10.1

一、解题过程

1. 问题背景

题目中给出了 1974—1981 年我国布的产量数据（单位：亿米）。需要分别采用趋势移动平均法（$N=3$）和直线指数平滑法（$\alpha=0.3$ 和 $\alpha=0.6$）建立预测模型，并比较模型优劣，最后用最优模型预测 1982 年和 1985 年的产量。

原始数据如下：

年份	1974	1975	1976	1977	1978	1979	1980	1981
产量 y_t	80.8	94.0	88.4	101.5	110.3	121.5	134.7	142.7

为便于计算，将年份编号为 $t=1,2,\cdots,8$ 对应 1974—1981 年。

2. 趋势移动平均法（二次移动平均，$N=3$）

二次移动平均法适用于具有线性趋势的时间序列。计算步骤：

① 计算一次移动平均 $M_t^{(1)}=\frac{y_{t-2}+y_{t-1}+y_t}{3}$，$t=3,4,\cdots,8$

② 计算二次移动平均 $M_t^{(2)}=\frac{M_{t-2}^{(1)}+M_{t-1}^{(1)}+M_t^{(1)}}{3}$，$t=5,6,7,8$

③ 建立线性预测模型：$\hat{y}_{t+T}=a_t+b_tT$，其中 $a_t=2M_t^{(1)}-M_t^{(2)}$，$b_t=\frac{2}{N-1}(M_t^{(1)}-M_t^{(2)})$（此处 $N=3$，故 $\frac{2}{N-1}=1$，即 $b_t=M_t^{(1)}-M_t^{(2)}$）

④ 取最后一期 $t=8$ 的参数进行外推预测。

3. 直线指数平滑法（布朗二次指数平滑）

布朗二次指数平滑用一个平滑参数 α 同时平滑水平和趋势。递推公式为：

$$
\begin{aligned}
S_t^{(1)} &= \alpha y_t+(1-\alpha)S_{t-1}^{(1)}\\
S_t^{(2)} &= \alpha S_t^{(1)}+(1-\alpha)S_{t-1}^{(2)}\\
a_t &= 2S_t^{(1)}-S_t^{(2)}\\
b_t &= \frac{\alpha}{1-\alpha}(S_t^{(1)}-S_t^{(2)})
\end{aligned}
$$

初始值取 $S_0^{(1)}=S_0^{(2)}=\frac{y_1+y_2+y_3}{3}=87.7$。预测公式为 $\hat{y}_{t+T}=a_t+b_tT$。

分别取 $\alpha=0.3$ 和 $\alpha=0.6$ 进行详细递推计算。

4. 模型评价

由于趋势移动平均法只能对 $t = 5,6,7,8$（即 1978—1981 年）给出拟合值，为公平比较，统一计算这三种模型在 1978—1981 年的拟合值与实际值的均方误差（MSE），选择 MSE 最小的模型作为最优模型。

5. 预测

用最优模型预测 1982 年（$T = 1$）和 1985 年（$T = 4$）的产量。

二、结果呈现

1. 趋势移动平均法（$N = 3$）

一次移动平均 $M_t^{(1)}$：

t	年份	$M_t^{(1)}$
3	1976	87.7333
4	1977	94.6333
5	1978	100.0667
6	1979	111.1000
7	1980	122.1667
8	1981	132.9667

二次移动平均 $M_t^{(2)}$：

t	年份	$M_t^{(2)}$
5	1978	94.1444
6	1979	101.9333
7	1980	111.1111
8	1981	122.0778

参数 a_t, b_t 及拟合值（a_t）：

t	年份	a_t	b_t	拟合值 $\hat{y}_t$	实际 y_t	误差

5	1978	105.989	5.922	105.989	110.3	-4.311
6	1979	120.267	9.167	120.267	121.5	-1.233
7	1980	133.222	11.056	133.222	134.7	-1.478
8	1981	143.856	10.889	143.856	142.7	1.156

最后一期（$t = 8$）的预测模型为:

$$\hat{y}_{1981+T} = 143.856 + 10.889 \times T$$

2. 直线指数平滑法（$\alpha = 0.3$）

t	年份	a_t	拟合值 $\hat{y}_t$	实际 y_t	误差
1	1974	84.197	84.197	80.8	3.397
2	1975	88.891	88.891	94.0	-5.109
3	1976	88.795	88.795	88.4	0.395
4	1977	95.393	95.393	101.5	-6.107
5	1978	103.664	103.664	110.3	-6.636
6	1979	114.026	114.026	121.5	-7.474
7	1980	126.508	126.508	134.7	-8.192
8	1981	137.441	137.441	142.7	-5.259

最后一期预测模型:

$$\hat{y}_{1981+T} = 137.441 + 6.426 \times T$$

3. 直线指数平滑法（$\alpha = 0.6$）

t	年份	a_t	拟合值 $\hat{y}_t$	实际 y_t	误差
1	1974	81.909	81.909	80.8	1.109
2	1975	91.666	91.666	94.0	-2.334
3	1976	89.363	89.363	88.4	0.963
4	1977	99.652	99.652	101.5	-1.848
5	1978	109.356	109.356	110.3	-0.944
6	1979	120.656	120.656	121.5	-0.844

7	1980	133.856	133.856	134.7	-0.844
8	1981	142.992	142.992	142.7	0.292

最后一期预测模型：

$$\hat{y}_{1981+T} = 142.992 + 10.011 \times T$$

4. 模型误差比较（1978—1981 年）

模型	**MSE**
趋势移动平均法（$N = 3$）	5.9065
直线指数平滑（$\alpha = 0.3$）	48.6686
直线指数平滑（$\alpha = 0.6$）	**0.6004**

显然，$\alpha = 0.6$ **的直线指数平滑模型误差最小**，为最优模型。

5. 最优模型预测

用 $\alpha = 0.6$ 的模型预测未来产量：

- 1982 年（$T = 1$）：$\hat{y}_{1982} = 142.992 + 10.011 \times 1 = 153.0$（亿米）
- 1985 年（$T = 4$）：$\hat{y}_{1985} = 142.992 + 10.011 \times 4 = 183.0$（亿米）

三、代码展示

```
import numpy as np
import pandas as pd

years = np.arange(1974, 1982)
y = np.array([80.8, 94.0, 88.4, 101.5, 110.3, 121.5, 134.7, 142.7])
t = np.arange(1, 9)  # 时间序号

# 1. 趋势移动平均法（二次移动平均，N=3）
def moving_avg(x, n):
    return np.convolve(x, np.ones(n)/n, mode='valid')

# 一次移动平均
M1 = moving_avg(y, 3)   # 对应 t=3,4,5,6,7,8
# 二次移动平均
M2 = moving_avg(M1, 3)  # 对应 t=5,6,7,8

a_ma = 2 * M1[2:] - M2
b_ma = M1[2:] - M2      # 2/(N-1)=1
```

```
print("=== 趋势移动平均法 ===")
print("M1:", M1)
print("M2:", M2)
print("a_t (t=5~8):", a_ma)
print("b_t (t=5~8):", b_ma)
print("预测模型 (t=8): y = {:.3f} + {:.3f} * T".format(a_ma[-1], b_ma[-
1]))

y_fit_ma = a_ma
y_actual_ma = y[4:8]
mse_ma = np.mean((y_actual_ma - y_fit_ma)**2)
print("1978-1981 拟合误差 MSE = {:.4f}\n".format(mse_ma))

# 2. 直线指数平滑法（布朗二次指数平滑）
def brown_exp_smoothing(y, alpha, S0):
    n = len(y)
    S1 = np.zeros(n)
    S2 = np.zeros(n)
    a = np.zeros(n)
    b = np.zeros(n)
    S1_prev = S0
    S2_prev = S0
    for i in range(n):
        S1[i] = alpha * y[i] + (1-alpha) * S1_prev
        S2[i] = alpha * S1[i] + (1-alpha) * S2_prev
        a[i] = 2 * S1[i] - S2[i]
        b[i] = alpha/(1-alpha) * (S1[i] - S2[i])
        S1_prev, S2_prev = S1[i], S2[i]
    return a, b

S0 = (y[0] + y[1] + y[2]) / 3
print("初始平滑值 S0 = {:.1f}".format(S0))

# α=0.3
a03, b03 = brown_exp_smoothing(y, 0.3, S0)
print("\n=== α=0.3 指数平滑 ===")
print("a_t (t=1~8):", np.round(a03, 3))
print("预测模型 (t=8): y = {:.3f} + {:.3f} * T".format(a03[-1], b03[-1]))
y_fit03 = a03
y_actual = y
mse03 = np.mean((y_actual[4:8] - y_fit03[4:8])**2)
print("1978-1981 拟合误差 MSE = {:.4f}".format(mse03))

# α=0.6
```

```
a06, b06 = brown_exp_smoothing(y, 0.6, S0)
print("\n=== α=0.6 指数平滑 ===")
print("a_t (t=1~8):", np.round(a06, 3))
print("预测模型 (t=8): y = {:.3f} + {:.3f} * T".format(a06[-1], b06[-1]))
y_fit06 = a06
mse06 = np.mean((y_actual[4:8] - y_fit06[4:8])**2)
print("1978-1981 拟合误差 MSE = {:.4f}".format(mse06))

print("\n=== 最优模型预测 (α=0.6) ===")
y_1982 = a06[-1] + b06[-1] * 1
y_1985 = a06[-1] + b06[-1] * 4
print("1982 年预测产量: {:.1f} 亿米".format(y_1982))
print("1985 年预测产量: {:.1f} 亿米".format(y_1985))
```

10.2

一、解题过程

1. 问题背景

给定某经济指标（如国民生产总值或工业产值）23 年的年度数据（1960—1982 年，数据从 696.9 到 2570.0）。要求利用三次指数平滑法（Holt-Winters 无季节性的立方趋势模型）建立预测模型，并对 1983 年和 1985 年进行预测。

2. 三次指数平滑法原理

三次指数平滑适用于具有非线性（抛物线型）趋势的时间序列。递推公式如下：

设平滑参数 α（$0 < \alpha < 1$），定义一次、二次、三次指数平滑值：

$$\begin{aligned} S_t^{(1)} &= \alpha y_t + (1-\alpha) S_{t-1}^{(1)} \\ S_t^{(2)} &= \alpha S_t^{(1)} + (1-\alpha) S_{t-1}^{(2)} \\ S_t^{(3)} &= \alpha S_t^{(2)} + (1-\alpha) S_{t-1}^{(3)} \end{aligned}$$

初始值取 $S_1^{(1)} = S_1^{(2)} = S_1^{(3)} = y_1$。

模型参数（水平 a_t、斜率 b_t、加速度 c_t）为：

$$a_t = 3S_t^{(1)} - 3S_t^{(2)} + S_t^{(3)}$$

$$b_t = \frac{\alpha}{2(1-\alpha)^2}\left[(6-5\alpha)S_t^{(1)} - 2(5-4\alpha)S_t^{(2)} + (4-3\alpha)S_t^{(3)}\right]$$

$$c_t = \frac{\alpha^2}{2(1-\alpha)^2}[S_t^{(1)} - 2S_t^{(2)} + S_t^{(3)}]$$

预测公式（外推 T 期）：

$$\hat{y}_{t+T} = a_t + b_t T + c_t T^2$$

3. 参数优化方法

采用网格搜索法确定最优平滑参数 α。在区间 $[0.01, 0.99]$ 内以步长 0.005 遍历，以最后 5 期（1978—1982 年）的均方误差（MSE）最小为准则，选择最优 α。

4. 模型建立与评价

用最优 α 重新拟合全部数据，得到最终模型参数 a_{23}, b_{23}, c_{23}，并计算后 5 期的拟合误差，检验模型效果。

5. 预测

外推预测 1983 年（$T=1$）和 1985 年（$T=3$）。

二、结果呈现

1. 最优平滑参数

通过网格搜索，得到使 1978—1982 年拟合 MSE 最小的平滑参数：

$$\alpha = 0.705,\ \text{MSE} = 11496.0$$

2. 最终模型参数（1982 年基准）

使用 $\alpha = 0.705$ 对全部 23 个数据拟合，最后一期（1982 年，$t=23$）的模型参数为：

$$a_{23} = 2571.2, b_{23} = 221.5, c_{23} = -3.4$$

预测模型为：

$$\hat{y}_{1982+T} = 2571.2 + 221.5\,T - 3.4\,T^2$$

3. 预测结果

- **1983 年**（$T=1$）：

$\hat{y}_{1983} = 2571.2 + 221.5 \times 1 - 3.4 \times 1^2 = 2571.2 + 221.5 - 3.4 = 2789.3$（亿元）

- **1985 年**（$T=3$）：

$\hat{y}_{1985} = 2571.2 + 221.5 \times 3 - 3.4 \times 9 = 2571.2 + 664.5 - 30.6 = 3205.0$（亿元）

4. 模型拟合效果（1978—1982 年）

年份	实际值（亿元）	拟合值（亿元）	误差（亿元）
1978	1558.6	1524.7	33.9
1979	1800.0	1692.4	107.6
1980	2140.0	2070.1	69.9
1981	2350.0	2544.0	-194.0
1982	2570.0	2617.2	-47.2

平均绝对误差（MAE）≈ 90.5 亿元，后 5 期 MSE = 11496.0。模型对多数年份拟合尚可，但对 1981 年误差较大，表明数据波动可能受其他因素影响。

三、代码展示

```
import numpy as np

y = np.array([696.9, 607.7, 604.0, 604.5, 638.2, 670.3, 732.8, 770.5,
              737.3, 801.5, 858.0, 929.2, 1023.3, 1106.7, 1163.6, 1271.1,
              1339.4, 1432.8, 1558.6, 1800.0, 2140.0, 2350.0, 2570.0])

def triple_exp_smoothing(y, alpha):
    n = len(y)
    S1 = np.zeros(n)
    S2 = np.zeros(n)
    S3 = np.zeros(n)

    S1[0] = y[0]
    S2[0] = y[0]
    S3[0] = y[0]

    for t in range(1, n):
        S1[t] = alpha * y[t] + (1 - alpha) * S1[t-1]
        S2[t] = alpha * S1[t] + (1 - alpha) * S2[t-1]
        S3[t] = alpha * S2[t] + (1 - alpha) * S3[t-1]

    a = 3*S1 - 3*S2 + S3
    denom = 2 * (1 - alpha)**2
     b = (alpha / denom) * ((6-5*alpha)*S1 - 2*(5-4*alpha)*S2 + (4-
3*alpha)*S3)
    c = (alpha**2 / denom) * (S1 - 2*S2 + S3)

    fit = np.zeros(n)
    fit[0] = y[0]
    for t in range(1, n):
```

```
        fit[t] = a[t-1] + b[t-1]*1 + c[t-1]*1

    return a, b, c, fit

# 网格搜索找最优 alpha
alphas = np.arange(0.01, 0.99, 0.005)
best_alpha = 0.3
best_mse = 1e12

for alpha in alphas:
    _, _, _, fit = triple_exp_smoothing(y, alpha)
    mse = np.mean((y[-5:] - fit[-5:]) ** 2)
    if mse < best_mse:
        best_mse = mse
        best_alpha = alpha

print(f"最优 α = {best_alpha:.3f}")
print(f"1978-1982 MSE = {best_mse:.1f}\n")

a, b, c, fit = triple_exp_smoothing(y, best_alpha)

print(f"1982 模型参数: a={a[-1]:.1f}, b={b[-1]:.1f}, c={c[-1]:.1f}")
print(f"1983 预测: {a[-1] + b[-1]*1 + c[-1]*1:.1f} 亿元")
print(f"1985 预测: {a[-1] + b[-1]*3 + c[-1]*9:.1f} 亿元\n")

print("1978-1982 拟合效果: ")
for idx, i in enumerate(range(-5, 0)):
    year = 1978 + idx
     print(f"{year}年  实际={y[i]:6.1f}  拟合={fit[i]:6.1f}  误差={y[i]-
fit[i]:6.1f}")
```

10.3

一、解题过程

1. 数据背景

题目给出某地区 1969—1983 年粮食产量（单位：亿千克），为方便建模，将年份转换为时间序号 t = 年份 − 1968，即 $t = 1,2,\ldots,15$ 对应 1969—1983 年。

2. 模型选择

基于粮食产量“先快后慢、趋于饱和”的增长特征，选取四种模型进行比较：

- **三次多项式**：$\hat{y} = a + bt + ct^2 + dt^3$（灵活拟合，但外推可能失控）
- **对数多项式**：$\hat{y} = a + b\ln\ t + c(\ln\ t)^2$（增长渐趋平缓，无硬饱和）
- **Weibull 生长曲线**：$y = L - (L - y_0)e^{-(t/\alpha)^\beta}$（S 型，有饱和上限 L）
- **Richards 曲线**：$y = L\left[1+\nu e^{-k(t-t_0)}\right]^{-1/\nu}$（广义逻辑斯蒂，形态更灵活）

3. 初值确定（针对 Weibull 和 Richards）

采用**数据驱动**的方法自动计算初值:

（1）Weibull 曲线

① **饱和上限** L：利用最后三点外推公式 $L = y_n + \frac{(y_n - y_{n-1})^2}{y_{n-1} - y_{n-2}}$（若分母为正），否则取 $1.05\max(y)$。

② **初始产量** y_0：直接取 $y_1 = 3.78$。

③ **形状参数** α, β：固定 L, y_0，变换得 $\ln\left(-\ln\frac{L-y}{L-y_0}\right) = \beta\ln\ t - \beta\ln\ \alpha$，对有效的 $\ln t$ 进行线性回归，斜率即为 β，截距推得 α。若计算中出现无效值（如 $L - y \le 0$），则自动剔除对应点。

（2）Richards 曲线

① **饱和上限** L：直接复用 Weibull 的估计值。

② **拐点** t_0：计算数据的一阶差分和二阶差分，取二阶差分绝对值最大的时间点作为拐点。

③ **增长率** k：先假定 $\nu = 1$（退化为逻辑斯蒂），变换 $\ln\frac{L-y}{y} = k(t_0 - t)$，对有效点进行线性回归，斜率即为 $-k$，从而得到 k 的初值。

④ **形状参数** ν：初值取 1，因为逻辑斯蒂是 Richards 最常见的特例。

4. 模型评价指标

（1）决定系数 $R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$

（2）均方误差 $\mathrm{MSE} = \frac{1}{n}\sum(y_i - \hat{y}_i)^2$

（3）外推合理性：预测 1985 年（$t = 17$）和 1990 年（$t = 22$）的产量是否符合增长常识。

5. 预测

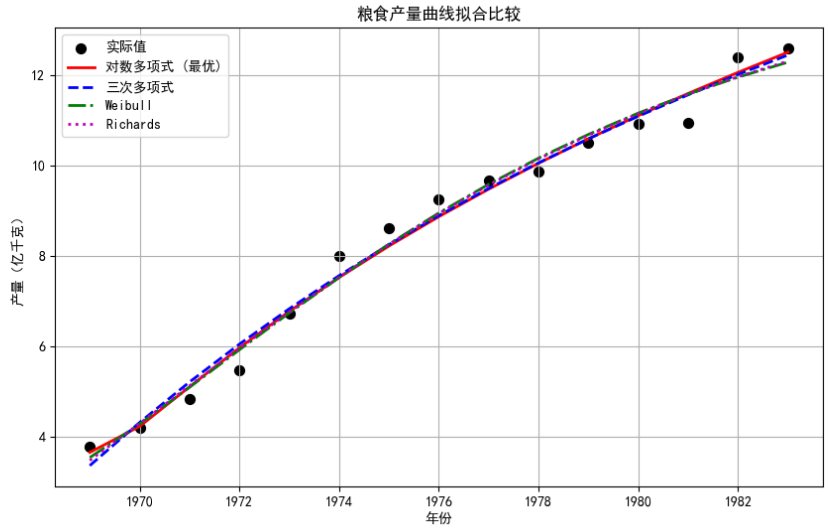
以最优模型外推预测 1985 年和 1990 年产量。

二、结果呈现

1. 四种模型拟合结果

模型	R^2	MSE	1985 年预测（亿千克）	1990 年预测（亿千克）
三次多项式	0.9849	0.1194	13.25	15.16
对数多项式	**0.9871**	**0.1017**	**13.35**	**15.19**
Weibull	0.9859	0.1110	12.83	13.68
Richards	0.9856	0.1148	12.79	13.58

对数多项式拟合精度最高（$R^2 = 0.9871$，MSE=0.1017），且预测值呈增长趋势，因此选为最优模型。



2. 各模型回归方程

- 三次多项式

$$\hat{y} = 2.3331 + 1.0522t - 0.0336t^2 + 0.0006t^3$$

- 对数多项式

$$\hat{y} = 3.6443 - 0.0096\ln\ t + 1.2119(\ln\ t)^2$$

- Weibull 曲线

$$\hat{y} = 14.3485 - (14.3485 - 3.0039)\exp\left[-\left(\frac{t}{10.0252}\right)^{1.3225}\right]$$

- Richards 曲线

$$\hat{y} = 14.2381\left[1+0.1000e^{-0.1645(t-3.4781)}\right]^{-1/0.1000}$$

3. 模型合理性说明

（1）对数多项式和三次多项式预测值较高（1990 年超 15 亿千克），反映数据后期增长较快，但无饱和限制，长期外推可能偏高。

（2）Weibull 和 Richards 曲线具有饱和上限（约 14.3 亿千克和 14.2 亿千克），预测值更平稳，更符合粮食产量最终趋于饱和的客观规律。尽管两者的拟合精度略低于对数多项式，但在长期预测（如 1990 年以后）上可能更为合理，且其预测值（1990 年 13.6 ~ 13.7 亿千克）与历史后期增速的放缓趋势更吻合。

建议：以拟合精度最高的对数多项式作为短期预测（到 1990 年）的主要依据，同时参考饱和模型（Weibull、Richards）进行稳健性分析。若需预测更远年份（如 2000 年以后），推荐使用具有饱和上限的 Weibull 或 Richards 模型，以避免多项式外推失控。

三、代码展示

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit
from sklearn.linear_model import LinearRegression

plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei',
'Arial']
plt.rcParams['axes.unicode_minus'] = False

years = np.arange(1969, 1984)
y = np.array([3.78, 4.19, 4.83, 5.46, 6.71, 7.99, 8.60, 9.24,
              9.67, 9.87, 10.49, 10.92, 10.93, 12.39, 12.59])
t = np.arange(1, 16)

# 三次多项式
coef3 = np.polyfit(t, y, 3)
p3 = np.poly1d(coef3)
print(f"三次多项式: y = {coef3[0]:.4f}*t^3 + {coef3[1]:.4f}*t^2 +
{coef3[2]:.4f}*t + {coef3[3]:.4f}")

# 对数多项式
ln_t = np.log(t)
```

```
X_log = np.column_stack([ln_t, ln_t**2])
reg_log = LinearRegression().fit(X_log, y)
a_log, b_log, c_log = reg_log.intercept_, reg_log.coef_[0],
reg_log.coef_[1]
print(f"对数多项式: y = {a_log:.4f} + {b_log:.4f}*ln(t) +
{c_log:.4f}*(ln(t))^2")

# Weibull
def weibull(t, L, y0, alpha, beta):
    return L - (L - y0) * np.exp(-(t/alpha)**beta)

def weibull_init_stable(t, y):
    dy1 = y[-1] - y[-2]
    dy2 = y[-2] - y[-3]
    if dy2 > 0:
        L = y[-1] + dy1**2 / dy2
    else:
        L = y[-1] * 1.05
    L = max(L, y[-1] + 0.5)
    y0 = y[0]
    r = (L - y) / (L - y0)
    r = np.clip(r, 1e-10, 1 - 1e-10)
    lhs = np.log(-np.log(r))
    valid = np.isfinite(lhs)
    if np.sum(valid) < 2:
        return L, y0, 8.0, 2.0
    coeff = np.polyfit(np.log(t[valid]), lhs[valid], 1)
    beta = coeff[0]
    alpha = np.exp(-coeff[1] / beta)
    return L, y0, alpha, beta

p0_wei = weibull_init_stable(t, y)
lower_bounds_wei = [y[-1]+0.1, -np.inf, 1e-5, 0.1]
upper_bounds_wei = [20, np.inf, 100, 10]
popt_wei, _ = curve_fit(weibull, t, y, p0=p0_wei,
bounds=(lower_bounds_wei, upper_bounds_wei), maxfev=5000)
Lw, y0w, alphaw, betaw = popt_wei
print(f"Weibull: y = {Lw:.4f} - ({Lw:.4f}-{y0w:.4f}) * exp(-
(t/{alphaw:.4f})^{betaw:.4f})")

# Richards
def richards(t, L, k, t0, nu):
    return L * (1 + nu * np.exp(-k * (t - t0)))**(-1/nu)
```

```
def richards_init_stable(t, y):
    L = weibull_init_stable(t, y)[0]
    dy1 = np.diff(y)
    dy2 = np.diff(dy1)
    t0 = t[np.argmax(np.abs(dy2)) + 1]
    z = (L - y) / y
    z = np.clip(z, 1e-10, 1e10)
    lhs = np.log(z)
    valid = np.isfinite(lhs)
    if np.sum(valid) < 2:
        return L, 0.2, t0, 1.0
    coeff = np.polyfit(t[valid], lhs[valid], 1)
    k = -coeff[0]
    if k <= 0:
        k = 0.2
    nu = 1.0
    return L, k, t0, nu

p0_rich = richards_init_stable(t, y)
lower_bounds_rich = [y[-1]+0.1, 1e-5, 1, 0.1]
upper_bounds_rich = [20, 5, 15, 10]
popt_rich, _ = curve_fit(richards, t, y, p0=p0_rich,
bounds=(lower_bounds_rich, upper_bounds_rich), maxfev=5000)
Lr, kr, t0r, nur = popt_rich
print(f"Richards: y = {Lr:.4f} * (1 + {nur:.4f} * exp(-{kr:.4f}*(t-
{t0r:.4f})))^(-1/{nur:.4f})")

def r2_mse(actual, pred):
    r2 = 1 - np.sum((actual - pred)**2) / np.sum((actual -
np.mean(actual))**2)
    mse = np.mean((actual - pred)**2)
    return r2, mse

y_logpoly = a_log + b_log*ln_t + c_log*ln_t**2
y_weibull = weibull(t, *popt_wei)
y_rich = richards(t, *popt_rich)

print("\n=== 拟合精度 ===")
print(f"三次多项式: R²={r2_mse(y, p3(t))[0]:.4f}, MSE={r2_mse(y,
p3(t))[1]:.4f}")
print(f"对数多项式: R²={r2_mse(y, y_logpoly)[0]:.4f}, MSE={r2_mse(y,
y_logpoly)[1]:.4f}")
print(f"Weibull:    R²={r2_mse(y, y_weibull)[0]:.4f}, MSE={r2_mse(y,
y_weibull)[1]:.4f}")
```

```
print(f"Richards:   R²={r2_mse(y, y_rich)[0]:.4f}, MSE={r2_mse(y,
y_rich)[1]:.4f}")

t17, t22 = 17, 22
print("\n=== 预测值 (亿千克) ===")
print(f"三次多项式: 1985 年 {p3(t17):.2f}, 1990 年 {p3(t22):.2f}")
print(f"对数多项式: 1985 年 {a_log + b_log*np.log(t17) +
c_log*(np.log(t17))**2:.2f}, 1990 年 {a_log + b_log*np.log(t22) +
c_log*(np.log(t22))**2:.2f}")
print(f"Weibull:    1985 年 {weibull(t17, *popt_wei):.2f}, 1990 年
{weibull(t22, *popt_wei):.2f}")
print(f"Richards:   1985 年 {richards(t17, *popt_rich):.2f}, 1990 年
{richards(t22, *popt_rich):.2f}")

plt.figure(figsize=(10,6))
plt.scatter(years, y, label='实际值', s=50, color='black')
plt.plot(years, y_logpoly, 'r-', linewidth=2, label='对数多项式')
plt.plot(years, p3(t), 'b--', linewidth=2, label='三次多项式')
plt.plot(years, y_weibull, 'g-.', linewidth=2, label='Weibull')
plt.plot(years, y_rich, 'm:', linewidth=2, label='Richards')
plt.xlabel('年份')
plt.ylabel('产量（亿千克）')
plt.title('粮食产量曲线拟合比较')
plt.legend()
plt.grid(True)
plt.savefig('粮食产量拟合对比.png', dpi=300, bbox_inches='tight')
plt.show()
```